Design Document: Logistics Platform (salter-mx)

> ***Version:*** *1.1.0* | ***Date:*** *2026-04-03*
>
> ***Changelog:***
>
> - `v1.1.0` *(2026-04-03): Modernized tech stack (Drizzle ORM, MapLibre GL JS, Inngest, Resend+React Email, Zod, Pino), offline resilience, idempotency keys, audit log schema, bulk import schema, Properties 46–50, MoSCoW release planning*
> - `v1.0.0` *(initial): First draft — architecture, Drizzle schema, 45 correctness properties*

Overview

salter-mx es un sistema operativo logístico full-stack construido sobre Next.js 15 (App Router) con TypeScript. El sistema gestiona el ciclo de vida completo de envíos: desde el registro y asignación a órdenes de transporte, hasta la entrega final con cumplimiento fiscal SAT/CFDI Carta Porte 3.1.

La arquitectura sigue un modelo monolítico modular: una sola unidad de despliegue con separación clara entre capas (UI, API Routes, lógica de negocio, acceso a datos). Esta decisión favorece la mantenibilidad para un equipo de una persona y elimina la complejidad operacional de microservicios.

Decisiones de Diseño Clave

- **Monolito modular sobre microservicios**: Reduce overhead operacional, facilita refactoring, y mantiene la testeabilidad sin infraestructura distribuida.
- **Drizzle ORM sobre raw SQL**: Type-safe, SQL-like syntax que preserva el conocimiento de SQL, auto-generación de migraciones con drizzle-kit, integración oficial con Neon, y sin runtime magic.
- **MapLibre GL JS para mapas**: Fork open-source (MIT) de Mapbox GL JS, GPU-accelerated, soporte móvil nativo, API-compatible con Mapbox para upgrade futuro sin cambios de código.
- **Inngest para jobs en background**: Serverless-native, persistencia de estado de jobs, retry automático con backoff exponencial, dashboard visual para debugging. Reemplaza colas en memoria. Maneja webhooks, notificaciones, importaciones masivas, y alertas críticas.
- **Resend + React Email para notificaciones**: Resend es el servicio de envío (TypeScript-first, free tier generoso). React Email permite escribir templates de email como componentes React, previsualizables en browser.
- **Zod para validación en todas las capas**: Schemas de validación en API routes (request bodies), variables de entorno, y parsing de respuestas de servicios externos (carriers, PAC). Garantiza type-safety en runtime, no solo en compilación.
- **Pino para logging estructurado**: Logger JSON de alto rendimiento para Node.js. Cada request recibe un correlationId único propagado a todos los logs. Niveles: info, warn, error, critical.
- **Idempotency keys para operaciones críticas**: Creación de órdenes y cambios de estado incluyen un clientId generado en el cliente. El servidor detecta y rechaza duplicados, garantizando exactamente-una-vez semántica incluso con reintentos por pérdida de conectividad.
- **Offline resilience para operadores móviles**: El cliente móvil bufferiza GPS pings y cambios de estado en localStorage/IndexedDB cuando no hay conectividad. Al reconectarse, sincroniza en orden con idempotency keys para evitar duplicados.
- **Auth.js v5 para autenticación**: Integración nativa con Next.js App Router, JWT sessions, extensible para OAuth futuro.

Release Planning

This design covers the full platform scope. Implementation follows MoSCoW prioritization defined in requirements.md.

Release	Scope	Key Design Components
Release 1 (MVP)	Must Have requirements (1,2,3,4,11,14,15)	Drizzle schema (core tables), Auth.js v5, API routes for core entities, Pino audit log, Inngest (import jobs + critical alerts), bulk import flow
Release 2	Should Have requirements (5,8,9,10,12,13)	SSE tracking + MapLibre map, CFDI builder + PAC adapter, full REST API + webhooks, dashboard + reports, Resend notifications, test suite
Release 3	Could Have requirements (6,7)	Route optimization, carrier adapters (Estafeta, DHL, FedEx, J&T)

Architecture

System Components

```
graph TB
    subgraph Client["Browser / Mobile"]
        UI[Next.js Pages & Components]
        MAP[Mapa de Rastreo - Leaflet]
    end

    subgraph Server["Next.js 15 App Router (Single Process)"]
        PAGES[Server Components / Pages]
        API[API Routes /api/v1/]
        BL[Business Logic Layer]
        REPO[Repository Layer]
        SSE[SSE Handler /api/v1/tracking/stream]
    end

    subgraph External["Servicios Externos"]
        PAC[PAC - Timbrado CFDI]
        CARRIERS[Transportistas: Estafeta / DHL / FedEx / J&T]
        GEOCODE[Geocodificación - Google Maps / Nominatim]
        EMAIL[Email - Resend / SendGrid]
        SMS[SMS - Twilio / Vonage]
    end

    subgraph Data["Datos"]
        PG[(PostgreSQL - Neon)]
        S3[Object Storage - S3/R2 para imágenes y PDFs]
    end

    UI --> PAGES
    UI --> API
    MAP --> SSE
    PAGES --> BL
```

```
    API --> BL
    BL --> REPO
    REPO --> PG
    BL --> PAC
    BL --> CARRIERS
    BL --> GEOCODE
    BL --> EMAIL
    BL --> SMS
    BL --> S3
```

Data Flow: Cambio de Estado de Envío

```
sequenceDiagram
    participant Client
    participant API as API Route
    participant BL as Business Logic
    participant DB as PostgreSQL
    participant WH as Webhook Service
    participant Email as Email Service

    Client->>API: PATCH /api/v1/envios/:id/estado
    API->>BL: updateEnvioEstado(id, nuevoEstado, userId)
    BL->>DB: BEGIN TRANSACTION
    BL->>DB: UPDATE envios SET estado = ?
    BL->>DB: INSERT INTO estado_historial
    BL->>DB: COMMIT
    BL->>WH: enqueueWebhook(event: "envio.estado_cambiado")
    BL->>Email: enqueueEmail(consignatario, nuevoEstado)
    BL-->>API: EnvioActualizado
    API-->>Client: 200 { data: EnvioActualizado }
    WH-->>ExternalSystem: POST webhook_url (async, con retry)
    Email-->>Consignatario: Email notificación (async, con retry)
```

Directory Structure

```
src/
├── app/
│   ├── (dashboard)/          # Route group: páginas autenticadas
│   │   ├── envios/
│   │   ├── ordenes/
│   │   ├── flota/
│   │   └── reportes/
│   ├── api/
│   │   └── v1/
│   │       ├── envios/
│   │       ├── ordenes/
│   │       ├── vehiculos/
│   │       ├── operadores/
│   │       ├── remitentes/
│   │       ├── consignatarios/
```

```
│   │       ├── tracking/
│   │       ├── cfdi/
│   │       └── webhooks/
│   └── tracking/[folio]/     # Página pública de rastreo
├── lib/
│   ├── db/
│   │   ├── index.ts          # Drizzle client setup con Neon
│   │   ├── schema/           # Drizzle schema definitions (un archivo por entidad)
│   │   └── migrations/       # Auto-generado por drizzle-kit
│   ├── repositories/         # Acceso a datos por entidad
│   ├── services/             # Lógica de negocio
│   ├── carriers/             # Abstracción de transportistas
│   ├── cfdi/                 # Generación XML Carta Porte
│   ├── notifications/        # Email y SMS
│   └── utils/                # Funciones puras (validaciones, formateo)
└── types/                    # TypeScript interfaces compartidas
```

Components and Interfaces

API Route Structure

Todos los endpoints bajo `/api/v1/` siguen la convención REST estándar. La autenticación de sesión (NextAuth) aplica a las rutas del dashboard; la autenticación por API Key aplica a los endpoints `/api/v1/`.

Método	Ruta	Descripción
GET	`/api/v1/envios`	Listar envíos (paginado)
POST	`/api/v1/envios`	Crear envío
GET	`/api/v1/envios/:id`	Obtener envío
PATCH	`/api/v1/envios/:id/estado`	Cambiar estado
GET	`/api/v1/envios/:folio/tracking`	Rastreo público (sin auth)
GET	`/api/v1/ordenes`	Listar órdenes
POST	`/api/v1/ordenes`	Crear orden
PATCH	`/api/v1/ordenes/:id/estado`	Cambiar estado de orden
GET	`/api/v1/vehiculos`	Listar vehículos
POST	`/api/v1/vehiculos`	Registrar vehículo
GET	`/api/v1/operadores`	Listar operadores
POST	`/api/v1/operadores`	Registrar operador
GET	`/api/v1/remitentes`	Buscar remitentes
POST	`/api/v1/remitentes`	Crear remitente
GET	`/api/v1/consignatarios`	Buscar consignatarios

POST	`/api/v1/consignatarios`	Crear consignatario
POST	`/api/v1/cfdi/generar`	Generar CFDI Carta Porte
POST	`/api/v1/cfdi/:uuid/cancelar`	Cancelar CFDI
GET	`/api/v1/tracking/stream`	SSE stream de ubicaciones
POST	`/api/v1/tracking/ubicacion`	Reportar ubicación GPS
POST	`/api/v1/tracking/sync`	Sync offline batch de eventos GPS
GET	`/api/docs`	Documentación OpenAPI 3.0
GET	`/api/v1/audit-log`	Consultar audit log (admin)
POST	`/api/v1/importacion`	Cargar archivo CSV/XLSX
GET	`/api/v1/importacion/:id`	Estado de importación en background
GET	`/api/v1/importacion/plantilla/:entidad`	Descargar plantilla CSV

Response Envelope

```typescript
// Todas las respuestas de la API siguen esta estructura
interface ApiResponse<T> {
  data: T | null;
  meta?: {
    page?: number;
    pageSize?: number;
    total?: number;
    [key: string]: unknown;
  };
  errors?: ApiError[];
}

interface ApiError {
  code: string;       // e.g. "FOLIO_DUPLICADO", "RFC_INVALIDO"
  message: string;    // Mensaje legible
  field?: string;     // Campo específico si aplica
}
```

Carrier Abstraction Layer

```typescript
// Strategy pattern para transportistas
interface CarrierAdapter {
  readonly name: string;
  readonly code: "ESTAFETA" | "DHL_MX" | "FEDEX_MX" | "JT_EXPRESS";

  getCotizacion(params: CotizacionParams): Promise<CotizacionResult>;
  generarGuia(params: GuiaParams): Promise<GuiaResult>;
  rastrearGuia(numeroGuia: string): Promise<TrackingResult>;
```

```
}

interface CotizacionParams {
  origen: Direccion;
  destino: Direccion;
  peso: number;       // kg
  dimensiones: Dimensiones;
  valorDeclarado: number; // MXN
}

interface CotizacionResult {
  costo: number;      // MXN
  tiempoEntregaDias: number;
  servicio: string;
  carrier: string;
}

// Registro de carriers disponibles
const carrierRegistry = new Map<string, CarrierAdapter>([
  ["ESTAFETA", new EstafetaAdapter(config.estafeta)],
  ["DHL_MX", new DhlMxAdapter(config.dhl)],
  ["FEDEX_MX", new FedexMxAdapter(config.fedex)],
  ["JT_EXPRESS", new JtExpressAdapter(config.jt)],
]);
```

Real-Time Tracking: Server-Sent Events

Se elige SSE sobre WebSockets porque el flujo de datos es unidireccional (servidor → cliente) para el mapa del dashboard. Los operadores reportan ubicación vía POST HTTP normal.

```
// GET /api/v1/tracking/stream
// Retorna un stream SSE con eventos de ubicación
// Evento: { vehiculoId, lat, lng, velocidad, timestamp, correlationId }

// POST /api/v1/tracking/ubicacion
// Body: { ordenId, lat, lng, velocidad, clientEventId, clientTimestamp }
// clientEventId: UUID generado en el dispositivo para idempotencia
// Almacena en DB (ignorando duplicados por clientEventId) y notifica SSE

// Offline resilience:
// El cliente móvil usa una cola local (IndexedDB via idb-keyval) para
// bufferizar eventos GPS cuando no hay conectividad.
// Al reconectarse, POST /api/v1/tracking/sync acepta un array de eventos
// ordenados por clientTimestamp, procesados en orden con idempotencia.
```

El estado compartido entre requests SSE se maneja con un simple `EventEmitter` en memoria (suficiente para un solo proceso). Para escalar horizontalmente en el futuro, se puede reemplazar con Redis Pub/Sub sin cambiar la interfaz.

Authentication: NextAuth.js v5

- **Sesiones**: JWT con expiración de 8 horas, almacenado en cookie HttpOnly

- **Estrategia**: Credentials provider (email + bcrypt password, cost factor 12)
- **RBAC**: El rol del usuario se incluye en el JWT payload y se verifica en middleware
- **API Key**: Header `X-API-Key` validado contra tabla `api_keys` en DB para endpoints `/api/v1/`
- **Rate Limiting**: Implementado en middleware de Next.js usando sliding window counter en PostgreSQL (tabla `rate_limit_counters`)

```
// Middleware de autenticación para API routes
// src/lib/middleware/auth.ts
export async function requireApiKey(req: Request): Promise<ApiKeyContext | ApiError>
export async function requireSession(req: Request): Promise<SessionContext |
ApiError>
export function requireRole(role: UserRole): Middleware
```

Background Jobs: Inngest

```
// Inngest maneja todos los jobs asincrónicos de la plataforma
// SDK: inngest (MIT), serverless-compatible con Next.js API Routes

// Jobs definidos:
// - envio/estado-cambiado: notificación email + SMS + webhook delivery
// - webhook/deliver: entrega con retry exponencial (1, 5, 15 min)
// - notificacion/send-email: envío via Resend con retry (2 intentos, 5 min)
// - notificacion/send-sms: envío via Twilio/Vonage con retry
// - cfdi/timbrar: timbrado con PAC con retry (3 intentos, 30 seg)
// - importacion/procesar: procesamiento de archivo CSV/XLSX en background
// - alerta/critical: notificación inmediata al admin ante error crítico
// - alerta/throttled: agrupación de errores repetidos (max 1 alerta / 10 min)

// Setup en Next.js:
// src/app/api/inngest/route.ts — endpoint que Inngest llama para ejecutar jobs
// src/lib/inngest/client.ts — Inngest client
// src/lib/inngest/functions/ — definición de cada job
```

Idempotency Keys

```
// Todas las operaciones de escritura críticas aceptan un header opcional:
// X-Idempotency-Key: <uuid-generado-por-cliente>
//
// El servidor almacena el resultado de la primera ejecución en la tabla
// idempotency_keys (key, response_status, response_body, created_at).
// Si llega un request con la misma key en las próximas 24 horas,
// retorna el resultado almacenado sin re-ejecutar la operación.
//
// Endpoints que requieren idempotency key:
// POST /api/v1/ordenes (creación de orden)
// PATCH /api/v1/envios/:id/estado (cambio de estado)
// POST /api/v1/cfdi/generar (generación de CFDI)
```

```
// POST /api/v1/tracking/ubicacion (reporte GPS)
// POST /api/v1/tracking/sync (sync offline batch)
```

Structured Logging: Pino

```
// src/lib/logger.ts
import pino from "pino";

export const logger = pino({
  level: process.env.LOG_LEVEL ?? "info",
  base: { service: "salter-mx" },
});

// Cada request recibe un correlationId (UUID v4) en middleware
// propagado via AsyncLocalStorage a todos los logs del request.
// Niveles de alerta:
// - critical: dispara Inngest job "alerta/critical" → email al admin
// - error: registrado en audit_log, visible en Dashboard
// - warn: registrado, no notifica
// - info: operaciones normales del negocio
```

CFDI / Carta Porte Generation

La generación de CFDI sigue este flujo:

```
sequenceDiagram
    participant BL as Business Logic
    participant Builder as CFDI Builder
    participant PAC as PAC API
    participant DB as PostgreSQL
    participant Storage as Object Storage

    BL->>Builder: buildCartaPorteXml(orden, empresa, config)
    Builder-->>BL: xmlString (sin timbrar)
    BL->>PAC: timbrar(xmlString, certificado)
    PAC-->>BL: { uuid, xmlTimbrado, sello, fechaTimbrado }
    BL->>DB: INSERT cfdi (uuid, xml, estado: "timbrado")
    BL->>Storage: upload(uuid + ".xml", xmlTimbrado)
    BL->>Builder: generatePdf(xmlTimbrado)
    Builder-->>BL: pdfBuffer
    BL->>Storage: upload(uuid + ".pdf", pdfBuffer)
    BL-->>Caller: CfdiResult
```

La generación de XML se implementa como funciones puras que construyen el árbol XML conforme al esquema SAT 3.1. Se usa la librería `xmlbuilder2` para construcción tipada del XML. El timbrado se abstrae detrás de una interfaz `PacAdapter` para soportar múltiples PACs (Finkok, SW Sapien, etc.).

```
interface PacAdapter {
  timbrar(xmlSinTimbrar: string): Promise<TimbradoResult>;
  cancelar(uuid: string, motivo: MotivoCancelacion, rfcEmisor: string):
```

```
Promise<CancelacionResult>;
}

interface TimbradoResult {
  uuid: string;
  xmlTimbrado: string;
  sello: string;
  fechaTimbrado: Date;
}
```

Data Models

Drizzle Schema

El esquema de base de datos se define en TypeScript usando Drizzle ORM. Cada entidad tiene su propio archivo en `src/lib/db/schema/` . Las migraciones SQL se auto-generan con `drizzle-kit generate` .

Setup del cliente (`src/lib/db/index.ts`)

```
import { neon } from "@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-http";
import * as schema from "./schema";

const sql = neon(process.env.DATABASE_URL!);
export const db = drizzle(sql, { schema });
```

> *Drizzle ORM usa `@neondatabase/serverless` como driver con el adaptador `drizzle-orm/neon-http` . Esto garantiza compatibilidad nativa con Neon y funciona también contra cualquier PostgreSQL estándar en tests (vía Testcontainers con el driver `node-postgres`).*

Tablas principales (`src/lib/db/schema/`)

```
// src/lib/db/schema/users.ts
import { pgTable, uuid, text, boolean, integer, timestamp } from "drizzle-orm/pg-
core";

export const users = pgTable("users", {
  id:             uuid("id").primaryKey().defaultRandom(),
  email:          text("email").notNull().unique(),
  passwordHash:   text("password_hash").notNull(),
  nombre:         text("nombre").notNull(),
  rol:            text("rol", { enum:
["administrador","coordinador","operador","cliente"] }).notNull(),
  activo:         boolean("activo").notNull().default(true),
  failedAttempts: integer("failed_attempts").notNull().default(0),
  lockedUntil:    timestamp("locked_until", { withTimezone: true }),
  createdAt:      timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:      timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
```

```
});

export const apiKeys = pgTable("api_keys", {
  id:        uuid("id").primaryKey().defaultRandom(),
  keyHash:   text("key_hash").notNull().unique(),
  nombre:    text("nombre").notNull(),
  userId:    uuid("user_id").notNull().references(() => users.id),
  activo:    boolean("activo").notNull().default(true),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
```

```
// src/lib/db/schema/remitentes.ts
import { pgTable, uuid, text, char, timestamp } from "drizzle-orm/pg-core";

const direccionColumns = {
  calle:     text("calle").notNull(),
  numExt:    text("num_ext").notNull(),
  numInt:    text("num_int"),
  colonia:   text("colonia").notNull(),
  municipio: text("municipio").notNull(),
  estado:    text("estado").notNull(),
  cp:        char("cp", { length: 5 }).notNull(),
  pais:      text("pais").notNull().default("MEX"),
};

export const remitentes = pgTable("remitentes", {
  id:        uuid("id").primaryKey().defaultRandom(),
  nombre:    text("nombre").notNull(),
  rfc:       text("rfc").notNull(),
  ...direccionColumns,
  telefono:  text("telefono").notNull(),
  email:     text("email").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});

export const consignatarios = pgTable("consignatarios", {
  id:        uuid("id").primaryKey().defaultRandom(),
  nombre:    text("nombre").notNull(),
  rfc:       text("rfc").notNull(),
  ...direccionColumns,
  telefono:  text("telefono").notNull(),
  email:     text("email").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});
```

```
// src/lib/db/schema/envios.ts
import { pgTable, uuid, text, numeric, timestamp } from "drizzle-orm/pg-core";
import { remitentes, consignatarios } from "./remitentes";
```

```
import { users } from "./users";

export const envios = pgTable("envios", {
  id:              uuid("id").primaryKey().defaultRandom(),
  folio:           text("folio").notNull().unique(),
  estado:          text("estado", {
                     enum:
["pendiente","recolectado","en_transito","en_reparto","entregado","no_entregado","canc
                   }).notNull().default("pendiente"),
  pesoKg:          numeric("peso_kg", { precision: 10, scale: 3 }).notNull(),
  largoCm:         numeric("largo_cm", { precision: 8, scale: 2 }),
  anchoCm:         numeric("ancho_cm", { precision: 8, scale: 2 }),
  altoCm:          numeric("alto_cm", { precision: 8, scale: 2 }),
  tipoContenido:   text("tipo_contenido").notNull(),
  valorDeclarado:  numeric("valor_declarado", { precision: 12, scale: 2
}).notNull().default("0"),
  comentarios:     text("comentarios"),
  remitenteId:     uuid("remitente_id").notNull().references(() => remitentes.id),
  consignatarioId: uuid("consignatario_id").notNull().references(() =>
consignatarios.id),
  ordenId:         uuid("orden_id"),  // FK a ordenes (definida en ordenes.ts para
evitar circular)
  createdBy:       uuid("created_by").notNull().references(() => users.id),
  createdAt:       timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:       timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const envioEstadoHistorial = pgTable("envio_estado_historial", {
  id:             uuid("id").primaryKey().defaultRandom(),
  envioId:        uuid("envio_id").notNull().references(() => envios.id),
  estadoAnterior: text("estado_anterior"),
  estadoNuevo:    text("estado_nuevo").notNull(),
  changedBy:      uuid("changed_by").notNull().references(() => users.id),
  lat:            numeric("lat", { precision: 10, scale: 7 }),
  lng:            numeric("lng", { precision: 10, scale: 7 }),
  notas:          text("notas"),
  createdAt:      timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const envioImagenes = pgTable("envio_imagenes", {
  id:         uuid("id").primaryKey().defaultRandom(),
  envioId:    uuid("envio_id").notNull().references(() => envios.id),
  url:        text("url").notNull(),
  filename:   text("filename").notNull(),
  sizeBytes:  integer("size_bytes").notNull(),
  createdAt:  timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});
```

```
// src/lib/db/schema/flota.ts
import { pgTable, uuid, text, numeric, integer, boolean, date, timestamp, char }
from "drizzle-orm/pg-core";

export const vehiculos = pgTable("vehiculos", {
  id:               uuid("id").primaryKey().defaultRandom(),
  placa:            text("placa").notNull().unique(),
  numeroEconomico:  text("numero_economico").notNull().unique(),
  tipo:             text("tipo", { enum:
["camioneta","camion_3_5t","camion_10t","trailer"] }).notNull(),
  capacidadKg:      numeric("capacidad_kg", { precision: 10, scale: 2 }).notNull(),
  capacidadM3:      numeric("capacidad_m3", { precision: 8, scale: 3 }).notNull(),
  anioModelo:       integer("anio_modelo").notNull(),
  estado:           text("estado", { enum:
["disponible","en_ruta","en_mantenimiento","inactivo"]
}).notNull().default("disponible"),
  configVehicular:  text("config_vehicular"),
  createdAt:        timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:        timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const operadores = pgTable("operadores", {
  id:            uuid("id").primaryKey().defaultRandom(),
  nombre:        text("nombre").notNull(),
  curp:          char("curp", { length: 18 }).notNull().unique(),
  licenciaNum:   text("licencia_num").notNull(),
  licenciaVence: date("licencia_vence").notNull(),
  telefono:      text("telefono").notNull(),
  rfc:           text("rfc"),
  activo:        boolean("activo").notNull().default(true),
  createdAt:     timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:     timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});
```

```
// src/lib/db/schema/ordenes.ts
import { pgTable, uuid, text, numeric, integer, date, timestamp } from "drizzle-
orm/pg-core";
import { vehiculos, operadores } from "./flota";
import { users } from "./users";

export const ordenes = pgTable("ordenes", {
  id:                       uuid("id").primaryKey().defaultRandom(),
  numeroOrden:              text("numero_orden").notNull().unique(),
  estado:                   text("estado", { enum:
["borrador","confirmada","en_ruta","completada","cancelada"]
}).notNull().default("borrador"),
```

```
  fechaEntregaComprometida: date("fecha_entrega_comprometida"),
  vehiculoId:               uuid("vehiculo_id").references(() => vehiculos.id),
  operadorId:               uuid("operador_id").references(() => operadores.id),
  pesoTotalKg:              numeric("peso_total_kg", { precision: 10, scale: 3
}).notNull().default("0"),
  distanciaKm:              numeric("distancia_km", { precision: 10, scale: 2 }),
  tiempoEstimadoMin:        integer("tiempo_estimado_min"),
  createdBy:                uuid("created_by").notNull().references(() => users.id),
  createdAt:                timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:                timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const ordenParadas = pgTable("orden_paradas", {
  id:         uuid("id").primaryKey().defaultRandom(),
  ordenId:    uuid("orden_id").notNull().references(() => ordenes.id),
  envioId:    uuid("envio_id").notNull(),  // FK a envios (evita circular)
  secuencia:  integer("secuencia").notNull(),
  lat:        numeric("lat", { precision: 10, scale: 7 }),
  lng:        numeric("lng", { precision: 10, scale: 7 }),
  completada: boolean("completada").notNull().default(false),
  createdAt:  timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});
```

```
// src/lib/db/schema/index.ts — re-exporta todo para el cliente Drizzle
export * from "./users";
export * from "./remitentes";
export * from "./envios";
export * from "./flota";
export * from "./ordenes";
export * from "./tracking";
export * from "./cfdi";
export * from "./webhooks";
export * from "./notificaciones";
export * from "./audit";
```

```
// src/lib/db/schema/audit.ts
import { pgTable, uuid, text, jsonb, timestamp, integer } from "drizzle-orm/pg-
core";

export const auditLog = pgTable("audit_log", {
  id:            uuid("id").primaryKey().defaultRandom(),
  correlationId: text("correlation_id").notNull(),
  nivel:         text("nivel", { enum: ["info","warn","error","critical"]
}).notNull(),
  evento:        text("evento").notNull(),
  userId:        uuid("user_id"),
  entidad:       text("entidad"),
```

```
  entidadId:     text("entidad_id"),
  payload:       jsonb("payload"),
  errorMessage:  text("error_message"),
  stackTrace:    text("stack_trace"),
  createdAt:     timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const idempotencyKeys = pgTable("idempotency_keys", {
  key:            text("key").primaryKey(),
  responseStatus: integer("response_status").notNull(),
  responseBody:   jsonb("response_body").notNull(),
  createdAt:      timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});

export const importaciones = pgTable("importaciones", {
  id:               uuid("id").primaryKey().defaultRandom(),
  userId:           uuid("user_id").notNull(),
  nombreArchivo:    text("nombre_archivo").notNull(),
  entidad:          text("entidad", { enum: ["remitentes","consignatarios","envios"]
}).notNull(),
  estado:           text("estado", { enum:
["pendiente","procesando","completado","fallido"] }).notNull().default("pendiente"),
  totalRegistros:   integer("total_registros"),
  registrosValidos: integer("registros_validos"),
  registrosError:   integer("registros_error"),
  reporteUrl:       text("reporte_url"),
  createdAt:        timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:        timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});
```

Indexes

Los índices se definen en los archivos de schema usando el helper `index` de Drizzle, o se incluyen en las migraciones SQL generadas por `drizzle-kit`. Los índices clave son:

- `envios`: folio, estado, remitente_id, consignatario_id, orden_id, created_at DESC
- `remitentes` / `consignatarios`: rfc, cp, nombre (GIN full-text en español)
- `ordenes`: numero_orden, estado, vehiculo_id
- `ubicaciones_gps`: (orden_id, created_at DESC), (vehiculo_id, created_at DESC)
- `webhook_deliveries`: (estado, proximo_intento) WHERE estado = 'pendiente'
- `notificaciones`: (estado, intentos) WHERE estado IN ('pendiente', 'fallido')

TypeScript Interfaces

```
// src/types/index.ts

export type EstadoEnvio =
  | "pendiente" | "recolectado" | "en_transito"
```

```
  | "en_reparto" | "entregado" | "no_entregado" | "cancelado";

export type EstadoOrden =
  | "borrador" | "confirmada" | "en_ruta" | "completada" | "cancelada";

export type EstadoVehiculo =
  | "disponible" | "en_ruta" | "en_mantenimiento" | "inactivo";

export type UserRole = "administrador" | "coordinador" | "operador" | "cliente";

export interface Direccion {
  calle: string;
  numExt: string;
  numInt?: string;
  colonia: string;
  municipio: string;
  estado: string;
  cp: string;
  pais: string;
  lat?: number;
  lng?: number;
}

export interface Envio {
  id: string;
  folio: string;
  estado: EstadoEnvio;
  pesoKg: number;
  largoCm?: number;
  anchoCm?: number;
  altoCm?: number;
  tipoContenido: string;
  valorDeclarado: number;
  comentarios?: string;
  remitenteId: string;
  consignatarioId: string;
  ordenId?: string;
  createdAt: Date;
  updatedAt: Date;
}

export interface Orden {
  id: string;
  numeroOrden: string;
  estado: EstadoOrden;
  fechaEntregaComprometida?: Date;
  vehiculoId?: string;
  operadorId?: string;
  pesoTotalKg: number;
  distanciaKm?: number;
  tiempoEstimadoMin?: number;
  createdAt: Date;
```

```
  updatedAt: Date;
}

export interface Vehiculo {
  id: string;
  placa: string;
  numeroEconomico: string;
  tipo: "camioneta" | "camion_3_5t" | "camion_10t" | "trailer";
  capacidadKg: number;
  capacidadM3: number;
  anioModelo: number;
  estado: EstadoVehiculo;
  configVehicular?: string;
}

export interface Operador {
  id: string;
  nombre: string;
  curp: string;
  licenciaNum: string;
  licenciaVence: Date;
  telefono: string;
  rfc?: string;
  activo: boolean;
}

export interface UbicacionGps {
  ordenId: string;
  vehiculoId: string;
  lat: number;
  lng: number;
  velocidadKmh?: number;
  createdAt: Date;
}
```

Docker Compose Setup

```
# docker-compose.yml
version: "3.9"
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: ${DATABASE_URL}
      NEXTAUTH_SECRET: ${NEXTAUTH_SECRET}
      NEXTAUTH_URL: http://localhost:3000
    depends_on:
      - postgres
    env_file: .env.local
```

```yaml
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: salter_dev
      POSTGRES_USER: salter
      POSTGRES_PASSWORD: salter_dev_pass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  postgres_test:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: salter_test
      POSTGRES_USER: salter
      POSTGRES_PASSWORD: salter_test_pass
    ports:
      - "5433:5432"

volumes:
  postgres_data:
```

Las migraciones se gestionan con `drizzle-kit` y se aplican explícitamente (no via `docker-entrypoint-initdb.d`):

```bash
# Generar migraciones SQL a partir del schema TypeScript
npx drizzle-kit generate

# Aplicar migraciones a la base de datos de desarrollo
npx drizzle-kit migrate

# Inspeccionar el schema actual de la DB
npx drizzle-kit studio

# Aplicar migraciones en CI / tests de integración
DATABASE_URL=postgres://salter:salter_test_pass@localhost:5433/salter_test npx
drizzle-kit migrate
```

La configuración de drizzle-kit se define en `drizzle.config.ts`:

```typescript
// drizzle.config.ts
import { defineConfig } from "drizzle-kit";

export default defineConfig({
  schema: "./src/lib/db/schema/index.ts",
  out: "./src/lib/db/migrations",
  dialect: "postgresql",
  dbCredentials: {
    url: process.env.DATABASE_URL!,
```

```
  },
});
```

Third-Party Services

Servicio	Propósito	Licencia	Alternativa
Neon (PostgreSQL)	Base de datos principal	Propietario (PostgreSQL estándar)	Cualquier PostgreSQL 16+
Drizzle ORM + drizzle-kit	ORM type-safe y migraciones	Apache 2.0	—
MapLibre GL JS	Mapas interactivos (tracking, rutas)	MIT	Mapbox GL JS (upgrade futuro)
Inngest	Background jobs, retry, durabilidad	MIT (SDK)	—
Resend	Envío de emails transaccionales	MIT (SDK)	Nodemailer + SMTP
React Email	Templates de email como componentes React	MIT	—
Twilio / Vonage	SMS	MIT (SDK)	Cualquier proveedor SMS REST
Google Maps Geocoding API	Geocodificación	Propietario	Nominatim (OSM, MIT)
Finkok / SW Sapien (PAC)	Timbrado CFDI	Propietario	Cualquier PAC SAT
Cloudflare R2 / AWS S3	Almacenamiento imágenes y PDFs	Propietario	MinIO (AGPL)
Zod	Validación de schemas en runtime	MIT	—
Pino	Logging estructurado JSON	MIT	—
Auth.js v5	Autenticación	ISC	—

Todos los servicios externos se acceden a través de interfaces abstractas (`EmailAdapter` , `SmsAdapter` , `GeocodingAdapter` , `PacAdapter` , `StorageAdapter`) para garantizar portabilidad.

Correctness Properties

A property is a characteristic or behavior that should hold true across all valid executions of a system — essentially, a formal statement about what the system should do. Properties serve as the bridge between human-readable specifications and machine-verifiable correctness guarantees.

Property 1: Round-trip de creación y lectura de entidades

For any entidad del sistema (envío, orden, vehículo, operador, remitente, consignatario) creada con un conjunto válido de campos, al recuperar esa entidad por su ID los campos almacenados deben ser equivalentes a los campos enviados en la creación.

Validates: Requirements 1.4, 2.1, 3.4, 4.1, 4.3

Property 2: Estado inicial correcto al crear entidad

For any envío recién creado, su estado debe ser `pendiente`. *For any* orden recién creada, su estado debe ser `borrador`. *For any* vehículo recién registrado, su estado debe ser `disponible`.

Validates: Requirements 1.2, 3.5, 4.2

Property 3: Historial de transición de estado

For any cambio de estado de un envío, el registro en `envio_estado_historial` debe contener el estado anterior, el estado nuevo, el ID del usuario que realizó el cambio, y una marca de tiempo no nula.

Validates: Requirements 1.3

Property 4: Rechazo de folio duplicado

For any folio que ya existe en la base de datos, un segundo intento de crear un envío con ese mismo folio debe retornar un error con código `FOLIO_DUPLICADO` y el envío no debe ser creado.

Validates: Requirements 1.5

Property 5: Límite de imágenes por envío

For any envío que ya tiene 5 imágenes adjuntas, intentar adjuntar una imagen adicional debe ser rechazado. *For any* imagen con formato distinto a JPG/PNG o tamaño mayor a 5 MB, el adjunto debe ser rechazado.

Validates: Requirements 1.6

Property 6: Paginación no excede límite

For any consulta paginada de envíos, el número de registros retornados en una sola página nunca debe exceder 50.

Validates: Requirements 1.7

Property 7: Validación de RFC

For any string que no cumpla el patrón RFC del SAT (12 caracteres alfanuméricos para persona moral, 13 para persona física con el formato correcto), la función de validación debe retornar `false`. *For any* RFC que sí cumpla el patrón, debe retornar `true`.

Validates: Requirements 2.2

Property 8: Validación de CP

For any string que no sea exactamente 5 dígitos numéricos, la función de validación de CP debe retornar `false`. *For any* string de exactamente 5 dígitos numéricos, debe retornar `true`.

Validates: Requirements 2.3

Property 9: Protección de entidades con dependencias activas

For any remitente o consignatario que tenga al menos un envío en estado distinto a `cancelado` o `entregado`, el intento de eliminación debe ser rechazado con un error que liste los envíos dependientes.

Validates: Requirements 2.5

Property 10: Creación de orden solo con envíos en estado pendiente

For any intento de crear una orden que incluya al menos un envío en estado distinto a `pendiente`, la operación debe ser rechazada. *For any* conjunto de envíos todos en estado `pendiente`, la creación de la orden debe ser exitosa.

Validates: Requirements 3.1

Property 11: Formato de número de orden

For any orden creada, su `numero_orden` debe cumplir el patrón `ORD-\d{6}-\d{5}` (ej. `ORD-202501-00042`), donde los primeros 6 dígitos corresponden al año y mes de creación.

Validates: Requirements 3.2

Property 12: Peso total de orden es suma de pesos de envíos

For any orden con N envíos asociados con pesos `p1, p2, ..., pN`, el campo `peso_total_kg` de la orden debe ser igual a `p1 + p2 + ... + pN`.

Validates: Requirements 3.3

Property 13: Cascada de estado al completar orden

For any orden con M envíos asociados, cuando la orden cambia a estado `completada`, todos sus M envíos deben cambiar a estado `entregado`.

Validates: Requirements 3.6

Property 14: Rechazo de envío ya asignado a orden activa

For any envío que ya está asignado a una orden en estado distinto a `cancelada`, intentar agregarlo a otra orden debe retornar un error que identifique la orden conflictiva.

Validates: Requirements 3.7

Property 15: Validación de capacidad al asignar vehículo

For any vehículo con estado distinto a `disponible`, la asignación a una orden debe ser rechazada. *For any* vehículo disponible cuya `capacidad_kg` sea menor al `peso_total_kg` de la orden, la asignación debe ser rechazada.

Validates: Requirements 4.4

Property 16: Alerta de licencia próxima a vencer

For any operador cuya `licencia_vence` sea una fecha dentro de los próximos 30 días (inclusive), debe aparecer en la lista de alertas de licencias próximas a vencer. *For any* operador con licencia que vence en más de 30 días, no debe aparecer en esa lista.

Validates: Requirements 4.5

Property 17: Cambio de estado de vehículo al asignar a orden confirmada

For any vehículo asignado a una orden que transiciona a estado `confirmada`, el estado del vehículo debe cambiar a `en_ruta`.

Validates: Requirements 4.6

Property 18: Persistencia de coordenadas GPS

For any reporte de ubicación GPS con coordenadas (lat, lng), velocidad, y timestamp, al consultar el historial de ubicaciones de esa orden los datos deben estar presentes y ser equivalentes a los reportados.

Validates: Requirements 5.2

Property 19: Coordenadas GPS en evento de entrega

For any cambio de estado de envío a `entregado` o `no_entregado` que incluya coordenadas GPS, el registro en `envio_estado_historial` debe contener esas coordenadas.

Validates: Requirements 5.4

Property 20: Alerta de vehículo sin señal

For any vehículo en estado `en_ruta` cuya última actualización de ubicación tenga más de 60 minutos de antigüedad, debe aparecer en la lista de alertas de vehículos sin señal.

Validates: Requirements 5.6

Property 21: Ruta sugerida es permutación válida de paradas

For any orden con N destinos, la ruta sugerida por el optimizador debe ser una permutación de esos N destinos (contiene exactamente los mismos destinos, sin duplicados ni omisiones).

Validates: Requirements 6.1

Property 22: Persistencia de ruta confirmada

For any ruta confirmada con paradas, distancia estimada y tiempo estimado, al recuperar la orden esos datos deben estar presentes y ser equivalentes a los confirmados.

Validates: Requirements 6.4

Property 23: Error descriptivo cuando geocodificación falla

For any dirección para la que el servicio de geocodificación retorna un error, el sistema debe retornar un error que identifique la dirección problemática (no un error genérico).

Validates: Requirements 6.5

Property 24: Persistencia de guía externa

For any guía generada con un transportista externo, al recuperar el envío asociado deben estar presentes el número de guía, código del carrier, costo, y fecha de generación.

Validates: Requirements 7.3

Property 25: Timeout de carrier externo retorna error sin bloquear

For any llamada a la API de un transportista externo que no responde dentro de 10 segundos, el sistema debe retornar un error descriptivo y la operación no debe bloquear otras operaciones del sistema.

Validates: Requirements 7.5

Property 26: XML CFDI válido y completo

For any orden completada con datos válidos de transportista, vehículo, operador, y mercancías, el XML generado debe: (a) ser válido contra el esquema XSD del SAT para Carta Porte 3.1, y (b) contener todos los campos requeridos (RFC transportista, placa, CURP/RFC operador, clave de producto SAT de cada mercancía).

Validates: Requirements 8.1, 8.2

Property 27: Round-trip de serialización XML CFDI

For any objeto `CartaPorteData` válido, serializar a XML y luego parsear el XML resultante debe producir un objeto equivalente al original.

Validates: Requirements 8.1, 13.12

Property 28: Persistencia de CFDI timbrado

For any CFDI timbrado exitosamente, al recuperarlo de la base de datos deben estar presentes el XML timbrado, la URL del PDF, el UUID fiscal, el sello, y la fecha de timbrado.

Validates: Requirements 8.4

Property 29: Manejo de fallo de timbrado

For any fallo retornado por el PAC durante el timbrado, el sistema debe almacenar el XML sin timbrar, registrar el mensaje de error del PAC, y el estado del CFDI debe ser `error` (no `timbrado`).

Validates: Requirements 8.5

Property 30: Validación de RFC contra lista SAT

For any RFC que no esté en el listado LCO/LRFC del SAT, el sistema debe rechazar su inclusión en un CFDI con un error que identifique el RFC inválido.

Validates: Requirements 8.7

Property 31: Autenticación requerida en endpoints API

For any request a `/api/v1/` sin header `X-API-Key` válido, el sistema debe retornar HTTP 401. *For any* request con API Key válida, debe ser procesado normalmente.

Validates: Requirements 9.2

Property 32: Estructura envelope en respuestas API

For any respuesta de cualquier endpoint bajo `/api/v1/`, el cuerpo JSON debe tener la estructura `{ data, meta?, errors? }` donde `data` es el resultado o `null` en caso de error.

Validates: Requirements 9.3

Property 33: Rate limiting por API Key

For any API Key que realiza más de 1000 requests en una ventana de 1 hora, el request que excede el límite debe recibir HTTP 429 con el header `Retry-After` indicando cuándo puede reintentar.

Validates: Requirements 9.4

Property 34: Entrega de webhooks y retry con espera exponencial

For any evento de negocio con un webhook endpoint configurado, el sistema debe encolar la entrega. *For any* webhook que falla, el sistema debe reintentar hasta 3 veces con intervalos de aproximadamente 1, 5, y 15 minutos entre intentos.

Validates: Requirements 9.6, 9.7

Property 35: Filtrado correcto en reportes

For any reporte generado con filtros de fecha, estado, transportista, remitente, o consignatario, todos los registros retornados deben cumplir todos los filtros aplicados, y ningún registro que cumpla los filtros debe ser omitido.

Validates: Requirements 10.2

Property 36: Cálculo correcto de tiempo promedio de entrega

For any conjunto de envíos entregados en un rango de fechas, el tiempo promedio calculado debe ser igual a la suma de los tiempos individuales de entrega dividida entre el número de envíos.

Validates: Requirements 10.3

Property 37: Ranking de consignatarios ordenado correctamente

For any conjunto de envíos, el ranking de consignatarios debe estar ordenado de mayor a menor volumen, contener máximo 10 entradas, y cada entrada debe mostrar el volumen correcto para ese consignatario.

Validates: Requirements 10.4

Property 38: Control de acceso por rol (RBAC)

For any usuario con rol `operador` o `cliente` que intenta acceder a un endpoint restringido a `administrador` o `coordinador`, el sistema debe retornar HTTP 403 y registrar el intento de acceso no autorizado.

Validates: Requirements 11.1, 11.6

Property 39: Hash seguro de contraseñas

For any contraseña almacenada en la base de datos, el valor almacenado debe ser un hash bcrypt verificable con la contraseña original y con un cost factor de al menos 12.

Validates: Requirements 11.3

Property 40: Bloqueo de cuenta tras fallos consecutivos

For any cuenta que acumula 5 fallos de autenticación consecutivos, el siguiente intento de autenticación (incluso con credenciales correctas) debe ser rechazado durante los siguientes 15 minutos.

Validates: Requirements 11.4

Property 41: Notificación email al cambiar estado de envío

For any cambio de estado de envío donde el evento está habilitado en la configuración de notificaciones, debe encolarse exactamente una notificación email al consignatario del envío.

Validates: Requirements 12.1, 12.2

Property 42: Persistencia de registro de notificaciones

For any notificación procesada (exitosa o fallida), debe existir un registro en la tabla `notificaciones` con timestamp, destinatario, tipo de evento, y estado de entrega.

Validates: Requirements 12.3

Property 43: Retry de notificaciones fallidas

For any notificación fallida, el sistema debe reintentar hasta 2 veces con intervalo de 5 minutos. Si persiste el fallo tras 2 reintentos, el estado debe quedar como `fallido` con el error registrado.

Validates: Requirements 12.4

Property 44: SMS condicional al cambiar a en_reparto o entregado

For any envío cuyo consignatario tiene número de teléfono registrado, al cambiar el estado a `en_reparto` o `entregado`, debe encolarse exactamente una notificación SMS.

Validates: Requirements 12.5

Property 45: Round-trip de respuestas de carriers

For any respuesta válida de la API de un transportista externo, parsear la respuesta y luego serializar el resultado debe producir un objeto equivalente al parseado originalmente.

Validates: Requirements 13.12

Property 46: Registro de eventos en audit log

For any operación de negocio principal (creación de envío, cambio de estado, generación de CFDI, autenticación), debe existir exactamente un registro en `audit_log` con el `correlationId` del request, nivel correcto, y timestamp no nulo.

Validates: Requirements 14.1, 14.8

Property 47: Notificación de error crítico al administrador

For any evento de nivel `critical`, debe encolarse exactamente un job Inngest `alerta/critical` que envíe email al administrador. *For any* secuencia de más de 5 eventos `critical` del mismo tipo en 10 minutos, debe encolarse exactamente un job `alerta/throttled` (no uno por evento).

Validates: Requirements 14.4, 14.7

Property 48: Dry-run no persiste datos

For any importación ejecutada en modo dry-run, el número de registros en las tablas `remitentes`, `consignatarios`, y `envios` debe ser idéntico antes y después de la operación.

Validates: Requirements 15.3

Property 49: Importación omite duplicados

For any registro en un archivo de importación cuyo RFC y nombre coincidan exactamente con un registro existente, ese registro debe aparecer en el reporte como "duplicado existente" y no debe crear un nuevo registro en la base de datos.

Validates: Requirements 15.8

Property 50: Idempotencia de operaciones críticas

For any operación con `X-Idempotency-Key` que ya fue procesada exitosamente, un segundo request con la misma key debe retornar el mismo `response_status` y `response_body` sin ejecutar la operación nuevamente, y el estado de la base de datos debe ser idéntico al estado tras la primera ejecución.

Validates: Requirements 14.8, 15.5

Error Handling

Error Response Format

Todos los errores siguen la estructura envelope estándar:

```
// HTTP 4xx / 5xx
{
  "data": null,
  "errors": [
    {
      "code": "FOLIO_DUPLICADO",
```

```
      "message": "El folio 'ABC-001' ya existe en el sistema",
      "field": "folio"  // opcional, para errores de validación de campo
    }
  ]
}
```

Error Codes Catalog

Código	HTTP	Descripción
`FOLIO_DUPLICADO`	409	El folio del envío ya existe
`RFC_INVALIDO`	422	El RFC no cumple el formato SAT
`CP_INVALIDO`	422	El CP no tiene 5 dígitos numéricos
`ENVIO_NO_PENDIENTE`	422	Se intentó agregar a orden un envío no pendiente
`ENVIO_EN_ORDEN_ACTIVA`	409	El envío ya pertenece a otra orden activa
`VEHICULO_NO_DISPONIBLE`	422	El vehículo no está en estado disponible
`CAPACIDAD_INSUFICIENTE`	422	El vehículo no tiene capacidad para el peso de la orden
`ENTIDAD_CON_DEPENDENCIAS`	409	No se puede eliminar entidad con registros dependientes
`CARRIER_TIMEOUT`	504	La API del transportista no respondió en 10 segundos
`CARRIER_ERROR`	502	La API del transportista retornó un error
`PAC_TIMBRADO_ERROR`	502	El PAC retornó error durante el timbrado
`RFC_NO_EN_SAT`	422	El RFC no está en el listado LCO/LRFC del SAT
`GEOCODIFICACION_FALLIDA`	422	No se pudo geocodificar la dirección indicada
`RATE_LIMIT_EXCEDIDO`	429	Se excedió el límite de 1000 req/hora
`NO_AUTENTICADO`	401	Request sin credenciales válidas
`NO_AUTORIZADO`	403	El usuario no tiene permisos para esta operación
`CUENTA_BLOQUEADA`	403	La cuenta está bloqueada por intentos fallidos
`IMAGEN_INVALIDA`	422	Formato o tamaño de imagen no permitido
`LIMITE_IMAGENES`	422	El envío ya tiene 5 imágenes adjuntas

External Service Failures

Todos los servicios externos (PAC, carriers, geocodificación, email, SMS) se llaman con:

- **Timeout explícito**: 10 segundos para carriers, 30 segundos para PAC
- **Circuit breaker**: Si un servicio falla 5 veces consecutivas, se marca como degradado y se retorna error inmediato sin esperar timeout
- **Retry en background**: Para operaciones asíncronas (webhooks, notificaciones), se usa retry con backoff exponencial

- **Fallback**: Para geocodificación, si Google Maps falla se intenta con Nominatim

Database Errors

- Errores de constraint (unique, foreign key) se mapean a códigos de error de negocio específicos
- Errores de conexión se retornan como HTTP 503 con mensaje genérico (sin exponer detalles internos)
- Todas las operaciones que modifican múltiples tablas se ejecutan en transacciones

Testing Strategy

Overview

La estrategia de testing sigue un enfoque dual: tests unitarios para lógica de negocio aislada y tests de propiedades para verificar invariantes universales. Ambos son complementarios y obligatorios.

Tooling

Herramienta	Propósito
Vitest	Test runner principal (unitarios e integración). Rápido, compatible con TypeScript nativo, API compatible con Jest.
fast-check	Property-based testing. Librería madura para TypeScript/JavaScript con generadores arbitrarios.
Playwright	Tests E2E. Soporte nativo para Next.js, headless, y CI.
Supertest	HTTP assertions para tests de integración de API Routes.
@testcontainers/postgresql	PostgreSQL efímero para tests de integración. Drizzle se conecta al contenedor usando el driver `node-postgres` (`drizzle-orm/node-postgres`), que es compatible con el mismo schema definido para Neon.

Test Layers

Unit Tests (`src/**/*.test.ts`)

Cubren funciones puras y módulos desacoplados de Next.js:

```
src/lib/utils/
  ├── rfc.test.ts          # Validación RFC (Properties 7)
  ├── cp.test.ts           # Validación CP (Property 8)
  ├── orden.test.ts        # Formato número orden, peso total (Properties 11, 12)
  └── cfdi-builder.test.ts # Generación XML, round-trip (Properties 26, 27)

src/lib/services/
  ├── envios.test.ts       # Lógica de negocio de envíos
  ├── ordenes.test.ts      # Lógica de órdenes, cascada de estado
  └── notificaciones.test.ts # Lógica de notificaciones
```

Objetivo: completar en < 30 segundos en CI. Sin base de datos, sin HTTP.

Integration Tests (`src/**/*.integration.test.ts`)

Cubren API Routes con base de datos real (PostgreSQL de test en Docker):

```
src/app/api/v1/
  ├── envios.integration.test.ts
  ├── ordenes.integration.test.ts
  ├── vehiculos.integration.test.ts
  ├── operadores.integration.test.ts
  ├── remitentes.integration.test.ts
  ├── consignatarios.integration.test.ts
  ├── cfdi.integration.test.ts
  └── tracking.integration.test.ts
```

Cada suite de integración:

1. Levanta PostgreSQL de test (Testcontainers o instancia Docker fija en puerto 5433)
2. Ejecuta migraciones
3. Resetea el schema antes de cada `describe` block
4. Verifica contratos HTTP: status codes, estructura envelope, headers

Property-Based Tests

Integrados en los tests unitarios usando `fast-check`. Cada propiedad del diseño se implementa como un test PBT con mínimo 100 iteraciones.

Configuración de tag por test:

```
// Ejemplo de property test
import fc from "fast-check";
import { describe, it } from "vitest";

describe("RFC Validation", () => {
  // Feature: logistics-platform, Property 7: Validación de RFC
  it("acepta RFCs válidos de persona moral (12 chars)", () => {
    fc.assert(
      fc.property(
        fc.stringMatching(/^[A-Z]{3}\d{6}[A-Z0-9]{3}$/),
        (rfc) => validateRfc(rfc) === true
      ),
      { numRuns: 100 }
    );
  });
});
```

E2E Tests (`e2e/`)

Cubren flujos críticos del usuario con Playwright:

```
e2e/
  ├── registro-envio.spec.ts      # Flujo completo de registro de envío
  ├── cambio-estado.spec.ts       # Cambio de estado con historial
  ├── generacion-orden.spec.ts    # Crear orden, asignar vehículo
  └── rastreo-publico.spec.ts     # Consulta pública de rastreo por folio
```

Los tests E2E se ejecutan contra el entorno completo levantado con `docker compose -f docker-compose.test.yml up` .

Test Commands

```
npm test                    # Unitarios + integración (determinista, CI)
npm run test:unit           # Solo unitarios (< 30s)
npm run test:integration    # Solo integración (requiere DB de test)
npm run test:e2e            # E2E con Playwright
npm run test:coverage       # Cobertura (objetivo: 80% en src/lib/)
```

Coverage Configuration

```
// vitest.config.ts
export default defineConfig({
  test: {
    coverage: {
      provider: "v8",
      include: ["src/lib/**/*.ts"],
      exclude: ["src/lib/db/migrations/**"],
      thresholds: {
        lines: 80,
        functions: 80,
        branches: 75,
      },
    },
  },
});
```

Property Test Reference Table

Cada propiedad del diseño se implementa en exactamente un test PBT:

Propiedad	Archivo de Test	Tag
P1: Round-trip creación/lectura	`repositories/*.test.ts`	`logistics-platform, Property 1`
P2: Estado inicial correcto	`services/envios.test.ts`	`logistics-platform, Property 2`
P3: Historial de transición	`services/envios.test.ts`	`logistics-platform, Property 3`
P4: Folio duplicado	`services/envios.test.ts`	`logistics-platform, Property 4`
P5: Límite de imágenes	`services/envios.test.ts`	`logistics-platform, Property 5`

P6: Paginación	`services/envios.test.ts`	`logistics-platform, Property 6`
P7: Validación RFC	`utils/rfc.test.ts`	`logistics-platform, Property 7`
P8: Validación CP	`utils/cp.test.ts`	`logistics-platform, Property 8`
P9: Protección dependencias	`services/remitentes.test.ts`	`logistics-platform, Property 9`
P10: Orden solo con pendientes	`services/ordenes.test.ts`	`logistics-platform, Property 10`
P11: Formato número orden	`utils/orden.test.ts`	`logistics-platform, Property 11`
P12: Peso total orden	`utils/orden.test.ts`	`logistics-platform, Property 12`
P13: Cascada completar orden	`services/ordenes.test.ts`	`logistics-platform, Property 13`
P14: Envío en orden activa	`services/ordenes.test.ts`	`logistics-platform, Property 14`
P15: Capacidad vehículo	`services/flota.test.ts`	`logistics-platform, Property 15`
P16: Alerta licencia	`services/flota.test.ts`	`logistics-platform, Property 16`
P17: Estado vehículo en_ruta	`services/flota.test.ts`	`logistics-platform, Property 17`
P18: Persistencia GPS	`services/tracking.test.ts`	`logistics-platform, Property 18`
P19: GPS en evento entrega	`services/tracking.test.ts`	`logistics-platform, Property 19`
P20: Alerta sin señal	`services/tracking.test.ts`	`logistics-platform, Property 20`
P21: Ruta es permutación válida	`services/rutas.test.ts`	`logistics-platform, Property 21`
P22: Persistencia ruta	`services/rutas.test.ts`	`logistics-platform, Property 22`
P23: Error geocodificación	`services/rutas.test.ts`	`logistics-platform, Property 23`
P24: Persistencia guía externa	`services/carriers.test.ts`	`logistics-platform, Property 24`

P25: Timeout carrier	`services/carriers.test.ts`	`logistics-platform, Property 25`
P26: XML CFDI válido y completo	`cfdi/builder.test.ts`	`logistics-platform, Property 26`
P27: Round-trip XML CFDI	`cfdi/builder.test.ts`	`logistics-platform, Property 27`
P28: Persistencia CFDI timbrado	`services/cfdi.test.ts`	`logistics-platform, Property 28`
P29: Fallo de timbrado	`services/cfdi.test.ts`	`logistics-platform, Property 29`
P30: RFC contra lista SAT	`services/cfdi.test.ts`	`logistics-platform, Property 30`
P31: Auth API Key	`middleware/auth.test.ts`	`logistics-platform, Property 31`
P32: Envelope respuestas	`api/*.integration.test.ts`	`logistics-platform, Property 32`
P33: Rate limiting	`middleware/ratelimit.test.ts`	`logistics-platform, Property 33`
P34: Webhooks y retry	`services/webhooks.test.ts`	`logistics-platform, Property 34`
P35: Filtrado reportes	`services/reportes.test.ts`	`logistics-platform, Property 35`
P36: Promedio entrega	`utils/reportes.test.ts`	`logistics-platform, Property 36`
P37: Ranking consignatarios	`utils/reportes.test.ts`	`logistics-platform, Property 37`
P38: RBAC	`middleware/auth.test.ts`	`logistics-platform, Property 38`
P39: Hash contraseñas	`services/auth.test.ts`	`logistics-platform, Property 39`
P40: Bloqueo cuenta	`services/auth.test.ts`	`logistics-platform, Property 40`
P41: Email al cambiar estado	`services/notificaciones.test.ts`	`logistics-platform, Property 41`
P42: Persistencia notificaciones	`services/notificaciones.test.ts`	`logistics-platform, Property 42`
P43: Retry notificaciones	`services/notificaciones.test.ts`	`logistics-platform, Property 43`

P44: SMS condicional	`services/notificaciones.test.ts`	`logistics-platform, Property 44`
P45: Round-trip respuestas carriers	`carriers/*.test.ts`	`logistics-platform, Property 45`
P46: Registro en audit log	`services/audit.test.ts`	`logistics-platform, Property 46`
P47: Notificación error crítico	`services/audit.test.ts`	`logistics-platform, Property 47`
P48: Dry-run no persiste datos	`services/importacion.test.ts`	`logistics-platform, Property 48`
P49: Importación omite duplicados	`services/importacion.test.ts`	`logistics-platform, Property 49`
P50: Idempotencia operaciones críticas	`middleware/idempotency.test.ts`	`logistics-platform, Property 50`